

Validity Analysis: HUMANEVAL

Target context: US Professional Software Developers — Python IDE Code Suggestion (HumanEval Assessment)

Overall risk: **HIGH**

Deployment Context

We are a US IDE vendor building a code suggestion feature for American developers. The tool proposes Python function implementations that are validated by running the user's test suite. We care about functional correctness: does the generated code compile, run, and pass all tests? We need to evaluate the LLM's ability to produce functionally correct Python code.

Dimension Scores

Dimension	Score	Rating	Confidence
Input Ontology ■	2	Significant gaps	high
Input Content ■	2	Significant gaps	high
Input Form ✓	5	Strong alignment	high
Output Ontology	3	Moderate gaps	high
Output Content ✓	4	Minor gaps	high
Output Form ✓	5	Strong alignment	high
Average	3.5		

■ = highest concern ✓ = strongest dimension

Dimension Key

Abbr.	Dimension	Definition
IO	Input Ontology	Whether the benchmark's test case categories cover the query types expected in deployment.
IC	Input Content	Whether individual datapoint content is culturally and contextually appropriate for the target region.
IF	Input Form	Whether the input signal encoding (text, audio, image parameters) matches deployment conditions.
OO	Output Ontology	Whether the benchmark's output categories and scoring criteria reflect regionally valid decision boundaries.
OC	Output Content	Whether ground-truth labels align with the judgments of regional stakeholders.
OF	Output Form	Whether the expected output modality matches regional deployment needs and accessibility.

Overall Summary

HumanEval has strong validity for the narrow slice of the US IDE deployment that matches its design — standalone algorithmic Python function synthesis from self-contained docstrings — and the input-form and output-form axes match the deployment exactly. However, the two HIGH-priority dimensions for this deployment (Input Ontology and Input Content) show substantial structural gaps. Elicitation responses and empirical dataset profiling converge on the same finding: library-grounded synthesis, class-method synthesis, async/await patterns, and repository-level context-grounded completions — confirmed as the majority real-world use cases — have zero coverage in the benchmark. The paper itself acknowledges this distribution mismatch [Q48]. The secondary correctness axes (security hygiene, style conformance) that

enterprise customers care about per elicitation are also absent from the scored ontology, though the primary pass/fail oracle aligns well with US test-driven development practice. Empirical evidence from RepoMasterEval [WEB-27] and the OOP benchmark [WEB-13] confirms these are real model-capability gaps, not just benchmark omissions — meaning HumanEval scores will systematically overestimate model utility for the deployment.

ID	Evidence
Q48	"In addition to standalone functions, Python code found on GitHub contains class implementations, configuration files, scripts, and even files used to store data. This code is seemingly unrelated to synthesizing functions from docstrings, and we hypothesize that the distribution mismatch reduces HumanEval performance." (p.7)
WEB-13	OOP benchmark (ACL Findings 2024) — all 23 evaluated LLMs perform significantly worse on class-level OOP tasks than on HumanEval/MBPP (https://arxiv.org/abs/2401.06628)
WEB-27	RepoMasterEval — GPT-4 scores drop significantly from HumanEval to repository-level benchmark, confirming the validity gap is empirical not theoretical (https://arxiv.org/pdf/2408.03519)

Practical Guidance

What This Benchmark Measures

HumanEval reliably measures a model's ability to synthesize standalone, module-level Python functions from self-contained English docstrings using only standard-library constructs. For the deployment's general-purpose algorithmic minority use case it provides high-fidelity signal; the input-form and output-form alignment (dimensions IF=5, OF=5) ensure the measurement modality matches IDE completion exactly, and the pass/fail oracle (OO=3 with strong primary alignment) matches the deployment's CI test-suite validation paradigm.

Construct Depth

The benchmark probes the algorithmic-reasoning subcomponent of code synthesis deeply, including unbiased sampling estimators [Q15], temperature-by-k optimization [Q34], and single-sample selection heuristics for autocomplete deployment [Q38]. However, depth is bounded by the input ontology (IO=2): performance on chained reasoning degrades exponentially [Q90, Q93], higher-level specifications are not supported [Q88], and the construct of 'professional Python code completion' is materially underrepresented — the benchmark cannot probe library-call correctness, class-hierarchy reasoning, async I/O, or cross-file context retrieval at all.

What Else You Need

For valid deployment-level assessment, HumanEval scores must be supplemented across four axes: (1) library-grounded synthesis via DS-1000 [WEB-7], BigCodeBench [WEB-11], or DSCodeBench [WEB-10] to address the IO/IC gap on third-party library calls; (2) repository-level context via SWE-bench Verified [WEB-17], CrossCodeEval [WEB-20], or RepoBench [WEB-21] to address the dominant deployment mode flagged in elicitation Q3; (3) class-method synthesis via OOP benchmark [WEB-13] or ClassEval to address the elicitation-confirmed class-hierarchy gap; (4) security hygiene via CyberSecEval [WEB-23] or CWEval [WEB-26] to address the elicitation A2 enterprise concerns. Async/await and domain-specific (fintech/healthtech/ML pipeline) evaluations have no public benchmark coverage identified and would require custom evaluation development.

ID	Evidence
Q3	"Furthermore, we find that repeated sampling from the model is a surprisingly effective strategy for producing working solutions to difficult prompts." (p.1)
Q15	"To evaluate pass@k, we generate $n \geq k$ samples per task (in this paper, we use $n = 200$ and $k \leq 100$), count the number of correct samples $c \leq n$ which pass unit tests, and calculate the unbiased estimator." (p.3)
Q34	"In particular, for a 679M parameter model, the optimal temperature for pass@1 is $T^* = 0.2$ and the optimal temperature for pass@100 is $T^* = 0.8$." (p.5)
Q38	"For instance, when the model is used as an autocomplete tool where a user provides a prompt, we do not have unit tests, but would like to return only a single completion to the user for evaluation so as to not overwhelm them." (p.5)
Q88	"Moreover, Codex struggles to parse through increasingly long and higher-level or system-level specifications." (p.10)

Q90	"We find that as the number of chained building blocks in the docstring increases, model performance decreases exponentially." (p.10)
Q93	"With each additional component, pass rate drops by roughly a factor of 2-3." (p.10)
WEB-7	DS-1000 — library-grounded data science problems exist as a supplementation target (https://arxiv.org/abs/2211.11501)
WEB-10	https://arxiv.org/abs/2505.15621
WEB-11	https://arxiv.org/abs/2406.15877
WEB-13	OOP benchmark (ACL Findings 2024) — all 23 evaluated LLMs perform significantly worse on class-level OOP tasks than on HumanEval/MBPP (https://arxiv.org/abs/2401.06628)
WEB-17	SWE-bench Verified — 500 human-validated repository-level Python problems addressing the project-context gap (https://openai.com/index/introducing-swe-bench-verified/)
WEB-20	CrossCodeEval — cross-file code completion benchmark targeting the project-context gap directly (https://arxiv.org/pdf/2310.11248)
WEB-21	https://github.com/Leolty/repobench
WEB-23	CyberSecEval — comprehensive insecure-code generation benchmark addressing security hygiene gap (~40% Copilot vulnerability finding) (https://arxiv.org/pdf/2312.04724)
WEB-26	CWEval — outcome-driven simultaneous functionality + security evaluation (https://arxiv.org/pdf/2501.08200)

Input Ontology (IO)

Score: 2/5 — Significant gaps | Confidence: high

Benchmark: HUMANEVAL | **Context:** US Professional Software Developers — Python IDE Code Suggestion (HumanEval Assessment)

Input Ontology definition: Whether the benchmark's test case categories cover the query types expected in deployment.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

HumanEval's task taxonomy is explicitly bounded to standalone function synthesis assessing 'language comprehension, reasoning, algorithms, and simple mathematics' [Q22], with the authors themselves acknowledging that 'Python code found on GitHub contains class implementations, configuration files, scripts, and even files used to store data. This code is seemingly unrelated to synthesizing functions from docstrings' [Q48]. Empirical dataset profiling confirms this gap is total: across 40 sampled problems, the only imports are `stdlib` (``typing``, ``math``), with zero examples involving classes, `async/await`, third-party libraries, or cross-file context. Per elicitation, library-grounded synthesis, class-method synthesis, `async/await`, and repository-context completions are the majority deployment use cases — all categorically absent from the benchmark. Because IO is HIGH priority and the omissions cover the dominant deployment categories, this dimension scores 2 (significant concerns with multiple concrete violations). Strengths in standalone algorithmic coverage prevent a score of 1.

ID	Evidence
Q22	"Programming tasks in the HumanEval dataset assess language comprehension, reasoning, algorithms, and simple mathematics." (p.4)
Q48	"In addition to standalone functions, Python code found on GitHub contains class implementations, configuration files, scripts, and even files used to store data. This code is seemingly unrelated to synthesizing functions from docstrings, and we hypothesize that the distribution mismatch reduces HumanEval performance." (p.7)

Strengths

- Provides well-distributed coverage of standalone algorithmic sub-categories (string manipulation, list ops, math, parsing) — the 'meaningful minority baseline' confirmed in elicitation [DATASET-D3, DATASET-D7, DATASET-D11]
- Tasks explicitly assess 'language comprehension, reasoning, algorithms, and simple mathematics' [Q22], which maps cleanly onto the general-purpose algorithmic deployment segment

ID	Evidence
Q22	"Programming tasks in the HumanEval dataset assess language comprehension, reasoning, algorithms, and simple mathematics." (p.4)
D3	<code>`def parse_music(music_string: str) -> List[int]:`</code> """ Input to this function is a string representing musical notes in a special ASCII format. """ — Domain-neutral string parsing, no library calls
D7	<code>`def correct_bracketing(brackets: str):`</code> """ brackets is a string of "(" and ")". return True if every opening bracket has a corresponding closing bracket. """ — Stack-based bracket matching — algorithmic, no external context
D11	<code>`def move_one_ball(arr):`</code> """...determine if it is possible to get an array sorted in non-decreasing order by performing...right shift operation...""" — Array rotation problem — algorithmic, interview-style

Checklist

Checklist item definitions:

ID	Assessment Question
----	---------------------

IO-1	Identify test case categories required for regional deployment
IO-2	Check if taxonomy omits regionally relevant categories
IO-3	Check if taxonomy includes irrelevant categories
IO-4	Document category gaps harming content validity

Checklist responses:

IO-1: Elicitation identifies four required categories: standalone algorithmic (minority), library-grounded synthesis (majority), class-method synthesis (common), async/await (common), and repository-level context-grounded completion (dominant real-world mode).

IO-2: Confirmed omissions across multiple critical categories. The benchmark covers only standalone function synthesis [Q1, Q22]; it omits library-grounded calls (zero pandas/NumPy/boto3 across sample [DATASET-D20, DATASET-D21]), class-method synthesis (no `class` definitions observed [DATASET-D6, DATASET-D9]), async/await patterns (no `async def` observed [DATASET-D5, DATASET-D19]), and repository-level context (every prompt is self-contained [DATASET-D1, DATASET-D12]). The paper itself flags the distribution mismatch with real Python code [Q48].

IO-3: Some interview-puzzle problems (Tribonacci, Collatz, hungry rabbit narratives) represent recreational mathematics that probes algorithmic cleverness rather than professional software engineering judgment [DATASET-D14, DATASET-D23, DATASET-D26]. These are not strictly irrelevant — they exercise reasoning — but they consume evaluation weight that does not transfer to enterprise code patterns.

IO-4: Content validity is materially harmed: the elicitation-confirmed dominant deployment categories (library-grounded, class-method, async, repo-context) are entirely absent. Empirical evidence from OOP benchmark [WEB-13] and RepoMasterEval [WEB-27] indicates these gaps reflect genuine model capability differences, not just benchmark omissions, so the missing coverage is consequential.

ID	Evidence
Q1	"We introduce Codex, a GPT language model finetuned on publicly available code from GitHub, and study its Python code-writing capabilities." (p.1)
Q22	"Programming tasks in the HumanEval dataset assess language comprehension, reasoning, algorithms, and simple mathematics." (p.4)
Q48	"In addition to standalone functions, Python code found on GitHub contains class implementations, configuration files, scripts, and even files used to store data. This code is seemingly unrelated to synthesizing functions from docstrings, and we hypothesize that the distribution mismatch reduces HumanEval performance." (p.7)
D1	`def has_close_elements(numbers: List[float], threshold: float) -> bool:` — Module-level standalone function, typed signature, no external dependencies
D5	`def decode_cyclic(s: str):` — takes as input string encoded with encode_cyclic function. Returns decoded string. — Cyclic string encoding/decoding — test uses `random` and `string` stdlib modules
D6	`def incr_list(l: list):` — Return list with elements incremented by 1. — Simple list comprehension — trivially simple, no domain context
D9	`def sort_array(array):` — ...sort the given array in ascending order if the sum(first index value, last index value) is odd, or sort it in descending order. — Conditional sorting — uses only stdlib `sorted`
D12	`def valid_date(date):` — ...The date should be in the format: mm-dd-yyyy — Date string validation — no `datetime` library used; manual parsing
D14	`def tri(n):` — Everyone knows Fibonacci sequence...Tribonacci sequence is defined by the recurrence: tri(1) = 3 — Custom mathematical sequence — invented recurrence, no real-world analogue
D19	`def by_length(arr):` — ...replace each digit by its corresponding name from "One", "Two"... — Digit-to-name mapping, hardcoded lookup — interview puzzle
D20	`import math` — Only stdlib `math` imported across all 40 examples — zero third-party library imports
D21	`from typing import List` — `typing` is the most commonly imported non-stdlib module across examples; no pandas/NumPy/requests/boto3
D23	`def eat(number, need, remaining):` — You're a hungry rabbit, and you already have eaten a certain number of carrots... — Whimsical narrative wrapper around simple arithmetic — interview-style

D26	<code>`def get_odd_collatz(n): ""The Collatz conjecture is a conjecture in mathematics...""`</code> — Collatz sequence — recreational mathematics, no enterprise relevance
WEB-7	DS-1000 — library-grounded data science problems exist as a supplementation target (https://arxiv.org/abs/2211.11501)
WEB-13	OOP benchmark (ACL Findings 2024) — all 23 evaluated LLMs perform significantly worse on class-level OOP tasks than on HumanEval/MBPP (https://arxiv.org/abs/2401.06628)
WEB-17	SWE-bench Verified — 500 human-validated repository-level Python problems addressing the project-context gap (https://openai.com/index/introducing-swe-bench-verified/)
WEB-27	RepoMasterEval — GPT-4 scores drop significantly from HumanEval to repository-level benchmark, confirming the validity gap is empirical not theoretical (https://arxiv.org/pdf/2408.03519)

Information Gaps

- Whether the remaining 124 unsampled problems contain any class, async, or third-party-library examples (sample of 40 found none, strongly suggesting no but not conclusive)

Remediation

Gap 1: Zero benchmark coverage of library-grounded synthesis (pandas, NumPy, requests, boto3), the majority deployment use case per elicitation A1, confirmed empirically [DATASET-D20]

Recommendation: Supplement HumanEval with DS-1000 [WEB-7] (1,000 problems across 7 data science libraries) and BigCodeBench [WEB-11] (1,140 tasks requiring multi-library function calls). Report scores side-by-side rather than substituting; the combined score is more deployment-predictive than either alone.

Gap 2: Zero coverage of class-method synthesis, despite elicitation confirming class hierarchies as a common deployment pattern; OOP benchmark [WEB-13] shows this is a genuine model weakness

Recommendation: Add the OOP benchmark (ACL Findings 2024) [WEB-13] and ClassEval to the evaluation suite. Expect substantially lower scores than HumanEval and weight them more heavily for enterprise codebases that are heavily class-based.

Gap 3: No async/await pattern coverage despite elicitation confirming async I/O as a common deployment scenario; no public benchmark identified

Recommendation: Develop a custom async-Python evaluation suite covering FastAPI handlers, `asyncio.gather` patterns, async context managers, and async generators. Until that exists, treat HumanEval as silent on async correctness and supplement with code review of async completions in production.

ID	Evidence
D20	<code>`import math`</code> — Only <code>stdlib`</code> imported across all 40 examples — zero third-party library imports
WEB-7	DS-1000 — library-grounded data science problems exist as a supplementation target (https://arxiv.org/abs/2211.11501)
WEB-11	https://arxiv.org/abs/2406.15877
WEB-13	OOP benchmark (ACL Findings 2024) — all 23 evaluated LLMs perform significantly worse on class-level OOP tasks than on HumanEval/MBPP (https://arxiv.org/abs/2401.06628)

Input Content (IC)

Score: 2/5 — Significant gaps | Confidence: high

Benchmark: HUMANEVAL | **Context:** US Professional Software Developers — Python IDE Code Suggestion (HumanEval Assessment)

Input Content definition: Whether individual datapoint content is culturally and contextually appropriate for the target region.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

HumanEval problems are deliberately context-free, hand-written, and domain-neutral [\[Q14, Q21\]](#), a design choice motivated by contamination avoidance that conflicts directly with the deployment's content needs. The elicitation answer to Q3 was explicit: 'Most real completions reference project-internal types, helper modules, ORM models, and domain-specific conventions across fintech, healthtech, and ML/data pipeline codebases. The function's meaning is often only recoverable from surrounding files.' Dataset profiling confirms zero domain-specific content: no Decimal arithmetic, no PHI patterns, no DataFrame operations, no FHIR/HL7 structures across 40 examples. Two minor locale-specific assumptions (European comma-decimal in HumanEval/137, US-only mm-dd-yyyy date in HumanEval/124) are noted but minor relative to the structural gap. Because IC is HIGH priority and the structural context-mismatch is total, this scores 2.

ID	Evidence
Q3	"Furthermore, we find that repeated sampling from the model is a surprisingly effective strategy for producing working solutions to difficult prompts." (p.1)
Q14	"Though not a guarantee for problem novelty, all problems were hand-written and not programmatically copied from existing sources." (p.3)
Q21	"It is important for these tasks to be hand-written, since our models are trained on a large fraction of GitHub, which already contains solutions to problems from a variety of sources." (p.4)

Strengths

- Hand-written problems avoid training-data contamination [\[Q14, Q21\]](#), making benchmark scores interpretable as genuine capability signals within the standalone algorithmic slice
- Self-contained docstrings provide clean, interpretable test items where success/failure can be unambiguously attributed [\[DATASET-D24, DATASET-D25\]](#)
- Content is written in standard American English with no localization barriers for the US developer population

ID	Evidence
Q14	"Though not a guarantee for problem novelty, all problems were hand-written and not programmatically copied from existing sources." (p.3)
Q21	"It is important for these tasks to be hand-written, since our models are trained on a large fraction of GitHub, which already contains solutions to problems from a variety of sources." (p.4)
D24	<code>`def below_threshold(l: list, t: int): """Return True if all numbers in the list l are below threshold t."""`</code> — One-liner predicate — extremely simple, no domain context
D25	<code>`def truncate_number(number: float) -> float: """ Given a positive floating point number...Return the decimal part of the number."""`</code> — <code>`return number % 1.0`</code> — trivially simple canonical solution

Checklist

Checklist item definitions:

ID	Assessment Question
IC-1	Do inputs require region-specific cultural/geographic/dialectal knowledge?
IC-2	Does culturally sensitive content align with target culture?
IC-3	Flag inputs assuming Western-specific knowledge
IC-4	Regional annotator recruitment for culturally sensitive instances
IC-5	Document content issues harming content validity

Checklist responses:

IC-1: Inputs require minimal region-specific knowledge — content is domain-neutral algorithmic puzzles in American English. Two minor exceptions: HumanEval/137 assumes European comma-as-decimal convention [DATASET-D15, DATASET-D38], and HumanEval/124 enforces US-centric mm-dd-yyyy format while marking ISO 8601 as invalid [DATASET-D37].

IC-2: Content is generally culturally compatible with the US deployment context. The benchmark is silent on enterprise domain conventions (fintech Decimal precision, healthtech PHI handling, ML pipeline patterns) but does not contain content that conflicts with US developer culture.

IC-3: Western-specific knowledge requirements are minimal; the benchmark is closer to culturally neutral than Western-biased. The mm-dd-yyyy date format assumption in HumanEval/124 [DATASET-D37] is a US convention but penalizes the ISO 8601 international standard.

IC-4: Not applicable in the cultural sense — the deployment population is US developers and the content language matches. The relevant 'representativeness' concern is domain coverage (fintech/healthtech/ML), which is documented as absent.

IC-5: Content validity is harmed via construct underrepresentation: the dominant content type (project-embedded, library-using, domain-specific code) has zero benchmark coverage [DATASET-D20, DATASET-D2]. The paper acknowledges underspecified prompts [Q63] and lower-quality docstrings [Q83] as known content issues.

ID	Evidence
Q63	"Some prompts underspecify the function that is implemented, in which case a perfectly valid solution may be wrongly penalized by the unit test." (p.8)
Q83	"While generating docstrings may be more forgiving because natural language syntax is less strict than code syntax, docstrings in our dataset may be lower quality because developers tend to devote less time to writing docstrings." (p.9)
D2	<code>`def below_zero(operations: List[int]) -> bool: """ You're given a list of deposit and withdrawal operations on a bank account that starts with zero balance."""`</code> — Bank account simulation — uses only built-in arithmetic, no pandas/SQL/ORM
D15	<code>`compare_one(1, "2,3") → "2,3"`</code> — Comma-as-decimal-separator handling — European locale convention embedded in task
D20	<code>`import math`</code> — Only <code>stdlib`</code> <code>`math`</code> imported across all 40 examples — zero third-party library imports
D37	<code>`assert candidate("2003-04-12") == False`</code> — Date in ISO format (yyyy-mm-dd) returns False — benchmark enforces mm-dd-yyyy only
D38	<code>`compare_one("5,1", "6") → "6"`</code> — Comma decimal separator treated as valid real number representation

Information Gaps

- Whether any of the unsampled 124 problems incidentally reference domain-specific content that would partially close the gap

Remediation

Gap 1: All benchmark problems are context-free; the dominant real-world completion mode involves project-internal types, ORM models, and helper modules per elicitation A3

Recommendation: Add SWE-bench Verified [WEB-17] for repository-level evaluation and CrossCodeEval [WEB-20] for cross-file completion. Treat HumanEval pass@1 as a lower-bound 'algorithmic competence' floor and SWE-bench/CrossCodeEval scores as the deployment-relevant ceiling.

Gap 2: Zero domain-specific content for fintech, healthtech, or ML pipeline codebases — major enterprise segments confirmed in deployment context

Recommendation: Commission domain-specific micro-benchmarks: fintech (Decimal arithmetic, idempotency patterns, audit logging), healthtech (FHIR/HL7 parsing, PHI de-identification), and ML pipelines (pandas vectorization, shape assertions, reproducibility seeds). DS-1000 partially covers ML pipelines but does not cover boto3 orchestration, PySpark, or domain-regulated patterns.

ID	Evidence
WEB-17	SWE-bench Verified — 500 human-validated repository-level Python problems addressing the project-context gap (https://openai.com/index/introducing-swe-bench-verified/)
WEB-20	CrossCodeEval — cross-file code completion benchmark targeting the project-context gap directly (https://arxiv.org/pdf/2310.11248)

Input Form (IF)

Score: 5/5 — Strong alignment | Confidence: high

Benchmark: HUMANEVAL | **Context:** US Professional Software Developers — Python IDE Code Suggestion (HumanEval Assessment)

Input Form definition: Whether the input signal encoding (text, audio, image parameters) matches deployment conditions.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

The input form is a near-exact match. Both the benchmark and the deployment present an English natural-language docstring plus a Python function signature as input, expecting a Python function body as output. Each HumanEval prompt 'consist[s] of a header, a signature, and a docstring' [\[Q28\]](#), identical to the IDE deployment scenario. Stop sequences ('\\nclass', '\\n\\n', etc.) [\[Q29\]](#) mirror typical IDE completion truncation. Dataset profiling confirms every sampled example follows this structure [\[DATASET-D1, DATASET-D21\]](#). IF is correctly flagged LOWER priority in elicitation — no modality, encoding, or representation gap exists.

ID	Evidence
Q28	"To compute pass@k, we assemble each HumanEval problem into a prompt consisting of a header, a signature, and a docstring, which is illustrated in Figure 2." (p.4)
Q29	"We sample tokens from Codex until we encounter one of the following stop sequences: '\\n\\n', '\\n\\n', '\\n\\n', '\\n\\n', or '\\n\\n', since the model will continue generating additional functions or statements otherwise." (p.4)
D1	`def has_close_elements(numbers: List[float], threshold: float) -> bool:` — Module-level standalone function, typed signature, no external dependencies
D21	`from typing import List` — `typing` is the most commonly imported non-stdlib module across examples; no pandas/NumPy/requests/boto3

Strengths

- Prompt structure (docstring + typed signature) is structurally identical to IDE completion input [\[Q28, DATASET-D1\]](#)
- Stop sequence handling [\[Q29\]](#) approximates IDE completion truncation
- Standard American English natural-language prompts match deployment population language
- Modern Python idioms (typing module, type annotations) are present [\[DATASET-D21\]](#)

ID	Evidence
Q28	"To compute pass@k, we assemble each HumanEval problem into a prompt consisting of a header, a signature, and a docstring, which is illustrated in Figure 2." (p.4)
Q29	"We sample tokens from Codex until we encounter one of the following stop sequences: '\\n\\n', '\\n\\n', '\\n\\n', '\\n\\n', or '\\n\\n', since the model will continue generating additional functions or statements otherwise." (p.4)
D1	`def has_close_elements(numbers: List[float], threshold: float) -> bool:` — Module-level standalone function, typed signature, no external dependencies
D21	`from typing import List` — `typing` is the most commonly imported non-stdlib module across examples; no pandas/NumPy/requests/boto3

Checklist

Checklist item definitions:

ID	Assessment Question
----	---------------------

IF-1	Compare signal distributions between source and target
IF-2	Check if regional infrastructure supports data capture specs
IF-3	Identify domain-specific form differences
IF-4	Document form mismatches harming external validity

Checklist responses:

IF-1: Signal distributions match: both contexts are UTF-8 Python source with English docstrings. No resolution, sampling rate, or sensor-parameter analogue exists for this domain.

IF-2: US developer infrastructure (VS Code, PyCharm, modern Python 3.x) [WEB-5, WEB-6] fully supports the benchmark's input format. No infrastructure mismatch.

IF-3: Both benchmark and deployment are text-to-Python-code. The benchmark's filtering of files with line length >100 or >1000 [Q24] is a training-data choice, not an evaluation-input mismatch.

IF-4: No form mismatches identified that would harm external validity along the input-form axis.

ID	Evidence
Q24	"We filtered out files which were likely auto-generated, had average line length greater than 100, had maximum line length greater than 1000, or contained a small percentage of alphanumeric characters." (p.4)
Q28	"To compute pass@k, we assemble each HumanEval problem into a prompt consisting of a header, a signature, and a docstring, which is illustrated in Figure 2." (p.4)
Q29	"We sample tokens from Codex until we encounter one of the following stop sequences: '\nclass', '\ndef', '\n#', '\nif', or '\nprint', since the model will continue generating additional functions or statements otherwise." (p.4)
WEB-5	VS Code dominant IDE (74% of professional developers) — supports docstring+signature completion format (https://survey.stackoverflow.co/2023)
WEB-6	Stack Overflow 2025 — VS Code and Visual Studio retain top spots (https://survey.stackoverflow.co/2025)

Output Ontology (OO)

Score: 3/5 — Moderate gaps | Confidence: high

Benchmark: HUMANEVAL | **Context:** US Professional Software Developers — Python IDE Code Suggestion (HumanEval Assessment)

Output Ontology definition: Whether the benchmark's output categories and scoring criteria reflect regionally valid decision boundaries.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

The primary output ontology — binary pass/fail on unit tests [\[Q77\]](#) — aligns well with the deployment's primary correctness criterion (user test-suite passage). The paper makes a principled case for functional correctness over match-based metrics [\[Q10, Q40\]](#), and test-driven development [\[Q12\]](#) mirrors enterprise CI/CD practice. However, elicitation A2 confirms that enterprise customers additionally care about security hygiene (eval, shell injection, hardcoded secrets) and style conformance (PEP 8, type annotations) as secondary acceptance criteria. The benchmark's scored ontology entirely omits these: cryptographic anti-patterns are explored only in the appendix [\[Q204-Q210\]](#), not as scored categories, and style/security are not scored at all. The paper acknowledges that test suites have 'limited but targeted coverage' [\[Q129\]](#) and that pass/fail misses intent misalignment [\[Q160, Q165\]](#). Score 3 reflects strong primary-axis alignment but a confirmed secondary-axis gap that matters for enterprise deployment (OO is MODERATE priority).

ID	Evidence
Q10	"More fundamentally, match-based metrics are unable to account for the large and complex space of programs functionally equivalent to a reference solution." (p.2)
Q12	"Perhaps the most convincing reason to evaluate functional correctness is that it is used by human developers to judge code. A framework known as test-driven development dictates that software requirements be converted into test cases before any implementation begins, and success is defined by a program that passes these tests." (p.2)
Q40	"Finally, we compute BLEU scores for all Codex-12B HumanEval samples (at temperature 0.8) against their reference solutions. For each problem, when we plot the distributions of BLEU scores for correct and incorrect solutions, we notice significant overlap (Figure 8). Since an incorrect solution is guaranteed to be functionally inequivalent to the reference solution, we conclude that improvements in BLEU score may not indicate improved rates of functional correctness in practice." (p.6)
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)
Q129	"Human developers often write test suites with limited but targeted coverage, but this does not always work well against an algorithm, highlighting the challenges of evaluating correctness of programs." (p.14)
Q160	"This caches out in predictions that the model will complete confused code with confused code, insecure code with insecure code (see G), or biased code with similarly biased code (see F), regardless of the model's capability to produce secure, unbiased, and high-quality code." (p.26)
Q165	"Based on this we conclude that we have identified misalignment in Codex models." (p.27)
Q204	"To study this phenomenon, we asked Codex to suggest code that would call cryptographic libraries to generate cryptographic contexts, and then evaluated whether any of these outputs were clearly insecure." (p.32)
Q205	"When tested on a standard series of prompts asking the models to call functions to produce RSA keys or AES contexts, we find that Codex models of varying sizes frequently use clearly insecure configurations (See Figure 15)." (p.32)
Q206	"We used 5 prompts across different libraries for RSA and AES based on Sonar Source's Python vulnerability database, and generated ~30k samples total." (p.32)
Q207	"We then removed some generated samples based on expected runtime errors, as different model sizes tend to vary in whether they produce code that runs." (p.32)
Q208	"RSA keys were considered improperly configured if they were shorter than 2048 bits." (p.32)
Q209	"AES contexts were considered improperly configured if they used the ECB cipher mode (see Menezes et al. (2018), p. 228)." (p.32)

Q210	"We chose these two tests to evaluate as targets because there is consensus among cryptography experts that these configurations generally should not be used, and these were reasonable to evaluate programmatically." (p.32)
------	--

Strengths

- Pass/fail unit-test oracle [Q77] aligns precisely with the deployment's primary validation criterion
- Functional correctness over BLEU/match metrics [Q10, Q40] reflects principled measurement of the construct most relevant to deployment
- Test-driven development framing [Q12] mirrors US enterprise CI/CD culture
- Paper explicitly identifies intent misalignment [Q160, Q165] and over-reliance [Q99, Q100] as failure modes relevant to enterprise deployment risk

ID	Evidence
Q10	"More fundamentally, match-based metrics are unable to account for the large and complex space of programs functionally equivalent to a reference solution." (p.2)
Q12	"Perhaps the most convincing reason to evaluate functional correctness is that it is used by human developers to judge code. A framework known as test-driven development dictates that software requirements be converted into test cases before any implementation begins, and success is defined by a program that passes these tests." (p.2)
Q40	"Finally, we compute BLEU scores for all Codex-12B HumanEval samples (at temperature 0.8) against their reference solutions. For each problem, when we plot the distributions of BLEU scores for correct and incorrect solutions, we notice significant overlap (Figure 8). Since an incorrect solution is guaranteed to be functionally inequivalent to the reference solution, we conclude that improvements in BLEU score may not indicate improved rates of functional correctness in practice." (p.6)
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)
Q99	"One of the key risks associated with using code generation models in practice is over-reliance on generated outputs." (p.11)
Q100	"Due to the limitations described above as well as alignment issues described below, Codex may suggest solutions that superficially appear correct but do not actually perform the task the user intended." (p.11)
Q160	"This caches out in predictions that the model will complete confused code with confused code, insecure code with insecure code (see G), or biased code with similarly biased code (see F), regardless of the model's capability to produce secure, unbiased, and high-quality code." (p.26)
Q165	"Based on this we conclude that we have identified misalignment in Codex models." (p.27)

Checklist

Checklist item definitions:

ID	Assessment Question
OO-1	Review output label categories for regional relevance
OO-2	Identify missing regionally specific categories
OO-3	Flag categories encoding non-regional values
OO-4	Consider stakeholder-driven taxonomy redesign
OO-5	Document taxonomy issues harming structural/content validity

Checklist responses:

OO-1: Primary label space (pass/fail) is regionally relevant — US enterprise CI/CD is test-driven [Q12]. Cryptographic anti-pattern thresholds (RSA <2048 bits, AES ECB mode) [Q208, Q209] reflect 'consensus among cryptography experts' [Q210] valid in the US context.

OO-2: Confirmed missing categories: security hygiene beyond appendix-level cryptography (eval, shell injection, SQL injection, hardcoded secrets), style conformance (PEP 8, type annotations), and performance constraints. Elicitation A2 identifies these as enterprise acceptance criteria not captured by pass/fail.

OO-3: No categories encode non-US values. The RSA/AES thresholds [Q208-Q210] are technical consensus standards, not culturally specific.

OO-4: Stakeholder-driven taxonomy redesign would benefit deployment use: adding security hygiene checks (informed by CyberSecEval [WEB-23, WEB-24] or CWEval [WEB-26]) and style/lint scoring would address documented enterprise needs.

OO-5: Structural validity is partially harmed: pass/fail correctly represents one axis of the construct but the paper itself notes match-based metrics 'are unable to account for the large and complex space of programs functionally equivalent to a reference solution' [Q10] and test suites have 'limited but targeted coverage' [Q129]. The benchmark cannot score security or style dimensions that matter to deployment.

ID	Evidence
Q10	"More fundamentally, match-based metrics are unable to account for the large and complex space of programs functionally equivalent to a reference solution." (p.2)
Q12	"Perhaps the most convincing reason to evaluate functional correctness is that it is used by human developers to judge code. A framework known as test-driven development dictates that software requirements be converted into test cases before any implementation begins, and success is defined by a program that passes these tests." (p.2)
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)
Q129	"Human developers often write test suites with limited but targeted coverage, but this does not always work well against an algorithm, highlighting the challenges of evaluating correctness of programs." (p.14)
Q160	"This caches out in predictions that the model will complete confused code with confused code, insecure code with insecure code (see G), or biased code with similarly biased code (see F), regardless of the model's capability to produce secure, unbiased, and high-quality code." (p.26)
Q208	"RSA keys were considered improperly configured if they were shorter than 2048 bits." (p.32)
Q209	"AES contexts were considered improperly configured if they used the ECB cipher mode (see Menezes et al. (2018), p. 228)." (p.32)
Q210	"We chose these two tests to evaluate as targets because there is consensus among cryptography experts that these configurations generally should not be used, and these were reasonable to evaluate programmatically." (p.32)
WEB-23	CyberSecEval — comprehensive insecure-code generation benchmark addressing security hygiene gap (~40% Copilot vulnerability finding) (https://arxiv.org/pdf/2312.04724)
WEB-24	CyberSecEval 2 — updated security benchmark with Instruct Prime variant (https://arxiv.org/pdf/2404.13161)
WEB-26	CWEval — outcome-driven simultaneous functionality + security evaluation (https://arxiv.org/pdf/2501.08200)

Requires Expert Verification

- Whether specific enterprise procurement criteria (e.g., SOC 2, FedRAMP-relevant security checks) translate into scoring requirements beyond what CyberSecEval covers

Remediation

Gap 1: Security hygiene (eval, shell injection, SQL injection, hardcoded secrets) and style conformance are enterprise acceptance criteria per elicitation A2 but absent from the scored output ontology; only RSA/AES patterns appear in appendix [Q204-Q210]

Recommendation: Add CyberSecEval [WEB-23] / CyberSecEval 2 [WEB-24] for security hygiene and CWEval [WEB-26] for simultaneous functionality+security evaluation. For style, integrate ruff/flake8/mypy checks into a custom test harness since no published style benchmark was identified.

ID	Evidence
Q204	"To study this phenomenon, we asked Codex to suggest code that would call cryptographic libraries to generate cryptographic contexts, and then evaluated whether any of these outputs were clearly insecure." (p.32)
Q205	"When tested on a standard series of prompts asking the models to call functions to produce RSA keys or AES contexts, we find that Codex models of varying sizes frequently use clearly insecure configurations (See Figure 15)." (p.32)

Q206	"We used 5 prompts across different libraries for RSA and AES based on Sonar Source's Python vulnerability database, and generated ~30k samples total." (p.32)
Q207	"We then removed some generated samples based on expected runtime errors, as different model sizes tend to vary in whether they produce code that runs." (p.32)
Q208	"RSA keys were considered improperly configured if they were shorter than 2048 bits." (p.32)
Q209	"AES contexts were considered improperly configured if they used the ECB cipher mode (see Menezes et al. (2018), p. 228)." (p.32)
Q210	"We chose these two tests to evaluate as targets because there is consensus among cryptography experts that these configurations generally should not be used, and these were reasonable to evaluate programmatically." (p.32)
WEB-23	CyberSecEval — comprehensive insecure-code generation benchmark addressing security hygiene gap (~40% Copilot vulnerability finding) (https://arxiv.org/pdf/2312.04724)
WEB-24	CyberSecEval 2 — updated security benchmark with Instruct Prime variant (https://arxiv.org/pdf/2404.13161)
WEB-26	CWEval — outcome-driven simultaneous functionality + security evaluation (https://arxiv.org/pdf/2501.08200)

Output Content (OC)

Score: 4/5 — Minor gaps | Confidence: high

Benchmark: HUMANEVAL | **Context:** US Professional Software Developers — Python IDE Code Suggestion (HumanEval Assessment)

Output Content definition: Whether ground-truth labels align with the judgments of regional stakeholders.

Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

Label correctness for the primary unit-test oracle is objective and not culturally contingent — a function either passes the tests or it does not, and annotator representativeness is largely irrelevant for executable verification. OC is correctly weighted LOWER in elicitation. The paper is sparse on annotator demographics for docstring evaluation [\[Q79\]](#) and alignment bug introduction [\[Q167\]](#), but for the dominant pass/fail labels this is not a validity concern. Minor concerns: HumanEval/124 enforces a US-centric date format as ground truth [\[DATASET-D37\]](#), the paper acknowledges some prompts are underspecified leading to 'wrongly penalized' valid solutions [\[Q63\]](#), and some test suites have weak edge-case coverage [\[DATASET-D18, DATASET-D19\]](#). These are noise-level concerns relative to the dimension's overall robustness.

ID	Evidence
Q63	"Some prompts underspecify the function that is implemented, in which case a perfectly valid solution may be wrongly penalized by the unit test." (p.8)
Q79	"Due to the time consuming nature of this process, we only grade 10 samples per problem, for a total of 1640 problems, from Codex-D-12B at temperature 0.8." (p.9)
Q167	"We believe this is unlikely to be a large factor here, as the GitHub dataset contains plenty of poor-quality code. The bugs are designed to be of the sort we'd expect to appear commonly in the dataset; code that compiles and often runs without errors but gives an incorrect answer. Examples include off-by-one errors or single-character typographic errors." (p.27)
D18	<code>`def unique_digits(x): """Given a list of positive integers x. return a sorted list of all elements that hasn't any even digit."""</code> — Number-theoretic filter on digit parity — no real-world application
D19	<code>`def by_length(arr): """...replace each digit by its corresponding name from "One", "Two"..."""</code> — Digit-to-name mapping, hardcoded lookup — interview puzzle
D37	<code>`assert candidate('2003-04-12') == False`</code> — Date in ISO format (yyyy-mm-dd) returns False — benchmark enforces mm-dd-yyyy only

Strengths

- Unit-test labels are objective and executable [\[Q77, DATASET-D36\]](#) — not subject to cultural annotator disagreement
- Cryptographic thresholds grounded in expert consensus [\[Q210\]](#) rather than annotator judgment
- Deterministic oracle [\[DATASET-D36, DATASET-D31\]](#) means inter-annotator agreement is structurally guaranteed for pass/fail

ID	Evidence
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)
Q210	"We chose these two tests to evaluate as targets because there is consensus among cryptography experts that these configurations generally should not be used, and these were reasonable to evaluate programmatically." (p.32)
D31	<code>`def check(candidate): from random import randint, choice import string`</code> — Test uses randomized inputs — valid but opaque for manual inspection
D36	<code>`assert candidate([1, 2, -4, 5, 6]) == True`</code> — Unit test verifies balance goes below zero — deterministic oracle

Checklist

Checklist item definitions:

ID	Assessment Question
OC-1	Do ground-truth labels reflect regional stakeholder perspectives?
OC-2	Assess potential annotator-regional population disagreement
OC-3	Review annotator demographics from documentation
OC-4	Consider label re-annotation by regional annotators
OC-5	Review aggregation methods for minority perspective erasure
OC-6	Document label issues harming convergent/external validity

Checklist responses:

OC-1: Ground truth labels (pass/fail on unit tests) reflect objective execution outcomes, not stakeholder perspective. They align with US enterprise expectations of test-driven correctness.

OC-2: Minimal annotator-population disagreement risk for pass/fail labels. The docstring-evaluation labels [Q79] involve subjective human grading but are not the primary benchmark metric.

OC-3: Annotator demographics are not documented in the paper for any annotation task. For pass/fail this is acceptable; for docstring grading (1640 problems hand-graded) [Q79] it is a documentation gap but does not materially affect deployment validity since deployment uses pass/fail.

OC-4: Re-annotation not needed for primary metric. Two specific labels could be revisited: HumanEval/124 ISO 8601 marked invalid [DATASET-D37] and HumanEval/137 comma-decimal convention [DATASET-D38] embed locale assumptions, but these are isolated cases.

OC-5: Aggregation is mechanical (test passage) — no minority-perspective erasure risk.

OC-6: Convergent and external validity for label correctness are largely preserved. The acknowledged underspecification issue [Q63] introduces noise but applies equally to deployment, where the user's own tests are the oracle.

ID	Evidence
Q63	"Some prompts underspecify the function that is implemented, in which case a perfectly valid solution may be wrongly penalized by the unit test." (p.8)
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)
Q79	"Due to the time consuming nature of this process, we only grade 10 samples per problem, for a total of 1640 problems, from Codex-D-12B at temperature 0.8." (p.9)
Q167	"We believe this is unlikely to be a large factor here, as the GitHub dataset contains plenty of poor-quality code. The bugs are designed to be of the sort we'd expect to appear commonly in the dataset; code that compiles and often runs without errors but gives an incorrect answer. Examples include off-by-one errors or single-character typographic errors." (p.27)
D37	`assert candidate('2003-04-12') == False` — Date in ISO format (yyyy-mm-dd) returns False — benchmark enforces mm-dd-yyyy only
D38	`compare_one("5,1", "6") → "6"` — Comma decimal separator treated as valid real number representation

Information Gaps

- Annotator demographics for docstring evaluation [Q79] are undocumented; not material for pass/fail deployment use

Output Form (OF)

Score: 5/5 — Strong alignment | Confidence: high

Benchmark: HUMANEVAL | Context: US Professional Software Developers — Python IDE Code Suggestion (HumanEval Assessment)

Output Form definition: Whether the expected output modality matches regional deployment needs and accessibility.

Each section below is followed by an evidence table. Three types of evidence are cited: [QN] — verbatim quotes extracted from the benchmark paper; [WEB-N] — web sources consulted during assessment; [DN] / [DATASET-DN] — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

Output form is an exact match. The benchmark produces Python code evaluated by test execution [Q77, Q15], identical to the deployment’s IDE-suggested function bodies validated by the user’s own test suite. The pass@k metric [Q15, Q17] is correctly formalized with an unbiased estimator. Temperature optimization for k [Q34] provides useful guidance for deployment configuration choices. Stop-sequence handling [Q29] approximates IDE truncation. OF is LOWER priority and shows no mismatch.

ID	Evidence
Q15	“To evaluate pass@k, we generate $n \geq k$ samples per task (in this paper, we use $n = 200$ and $k \leq 100$), count the number of correct samples $c \leq n$ which pass unit tests, and calculate the unbiased estimator.” (p.3)
Q17	“One may be tempted to estimate pass@k with $1 - (1 - p_{\blacksquare})^k$ where $p_{\blacksquare}$ is the empirical estimate of pass@1, but we show that it is biased in Appendix A.” (p.3)
Q29	“We sample tokens from Codex until we encounter one of the following stop sequences: ‘\nclass’, ‘\ndef’, ‘\n#’, ‘\nif’, or ‘\nprint’, since the model will continue generating additional functions or statements otherwise.” (p.4)
Q34	“In particular, for a 679M parameter model, the optimal temperature for pass@1 is $T^* = 0.2$ and the optimal temperature for pass@100 is $T^* = 0.8$.” (p.5)
Q77	“When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests.” (p.9)

Strengths

- Python source code output evaluated by test execution [Q77] — identical modality to deployment
- Unbiased pass@k estimator [Q15] correctly handles sampling variance, supporting reliable measurement
- Temperature-by-k optimization [Q34] provides actionable guidance for IDE deployment sampling configuration
- Single-sample selection heuristics [Q37, Q38, Q39] explicitly address the IDE-autocomplete deployment scenario

ID	Evidence
Q15	“To evaluate pass@k, we generate $n \geq k$ samples per task (in this paper, we use $n = 200$ and $k \leq 100$), count the number of correct samples $c \leq n$ which pass unit tests, and calculate the unbiased estimator.” (p.3)
Q34	“In particular, for a 679M parameter model, the optimal temperature for pass@1 is $T^* = 0.2$ and the optimal temperature for pass@100 is $T^* = 0.8$.” (p.5)
Q37	“From a practical perspective, we are also interested in the setting where we must select a single sample from k samples without having access to an oracle.” (p.5)
Q38	“For instance, when the model is used as an autocomplete tool where a user provides a prompt, we do not have unit tests, but would like to return only a single completion to the user for evaluation so as to not overwhelm them.” (p.5)
Q39	“Inspired by similar work in language modeling, we find that choosing the sample with the highest mean token log probability outperforms evaluating a random sample, while choosing the sample based on sum log probability can perform slightly worse than picking randomly.” (p.5)
Q77	“When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests.” (p.9)

Checklist

Checklist item definitions:

ID	Assessment Question
OF-1	Does output modality match regional deployment needs?
OF-2	Assess text-to-speech availability for speech output
OF-3	Consider literacy rates and accessibility requirements
OF-4	Document form mismatches harming external validity

Checklist responses:

OF-1: Output modality (Python code) exactly matches IDE deployment needs.

OF-2: Not applicable — no speech output requirement in this deployment.

OF-3: Not applicable — target population is professional software developers; literacy is not a constraint.

OF-4: No form mismatches identified.

ID	Evidence
Q15	"To evaluate pass@k, we generate $n \geq k$ samples per task (in this paper, we use $n = 200$ and $k \leq 100$), count the number of correct samples $c \leq n$ which pass unit tests, and calculate the unbiased estimator." (p.3)
Q38	"For instance, when the model is used as an autocomplete tool where a user provides a prompt, we do not have unit tests, but would like to return only a single completion to the user for evaluation so as to not overwhelm them." (p.5)
Q77	"When we benchmark our code generation models, we measure pass@k on the HumanEval dataset, where correctness is defined by passing a set of unit tests." (p.9)